

sdmx

June 6, 2026

compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single
filename	sdmx.dsp
name	sdmx
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2
seam.stereophony.lib/author	Giuseppe Silvi
seam.stereophony.lib/license	CC3
seam.stereophony.lib/name	SEAM Stereophony Library
seam.stereophony.lib/version	0.3

This document provides a mathematical description of the Faust program text stored in the `sdmx.dsp` file. See the notice in Section 3 (page 2) for details.

1 Mathematical definition of process

The *sdmx* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 2]$ such that

$$y_1(t) = 0.707106781186547 \cdot (x_1(t) + x_2(t))$$

$$y_2(t) = 0.707106781186547 \cdot (x_1(t) - x_2(t))$$

2. Input signals x_i for $i \in [1, 2]$

2 Block diagram of process

The block diagram of `process` is shown on Figure 1 (page 2).

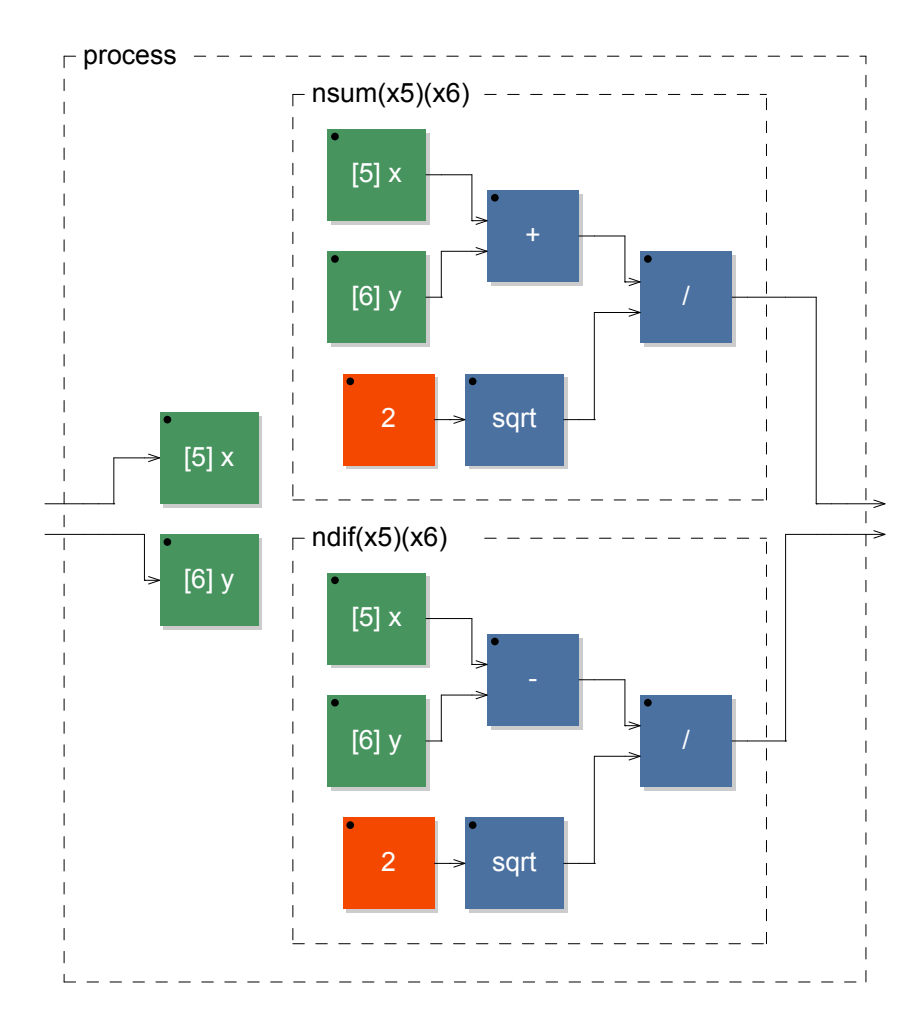


Figure 1: Block diagram of `process`

3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_S sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis appli-

cations. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).

- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `sdmx-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the L^AT_EX source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `sdmx.dsp`

```
1 import("seam.lib");
2 // Canonical DSP: sst.sdmx in seam.stereophony.lib
3 process = sst.sdmx;
```

Listing 2: `seam.lib`

```
1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
```

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

```

9  san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 // ### author specific literature
25 // sds = library("seam.discipio.lib");
26 smg = library("seam.gerzon.lib");
27 // sln = library("seam.nono.lib");
28 // sgn = library("seam.nottoli.lib");
29 sjm = library("seam.moorer.lib");
30 scr = library("seam.roads.lib");
31 sms = library("seam.schroeder.lib");
32 sfv = library("seam.freeverb.lib");
33
34 sff = library("seam.ffunctions.lib");
35
36 // ### live electronics stuff
37 // sgu = library("seam.gui.lib");
38 // shw = library("seam.hardware.lib");
39 // smx = library("seam.mixer.lib");
40
41 // ### instrument specific literature
42 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1  //##### stdfaust.lib #####
2  // The purpose of this library is to give access to all the Faust standard libraries
3  // through a series of environments.
4  //#####
5
6  aa = library("aanl.lib");
7  sf = library("all.lib");
8  an = library("analyzers.lib");
9  ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");

```

```

28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.stereophony.lib

```

1  declare name "SEAM Stereophony Library";
2  declare version "0.3";
3  declare author "Giuseppe Silvi";
4  declare license "CC3";
5  //
6  // SEAM Stereophony Library
7  //
8  // Functions for stereophonic encoding, decoding, and spatial manipulation.
9  // Based on the sum-and-difference principle introduced by Alan Blumlein
10 // (UK patent 394325, 1933) and extended by Michael Gerzon.
11 //
12 //
13 //
14 import("seam.lib");
15 //
16 sst = library("seam.stereophony.lib");
17 //
18 //          Sum-and-difference matrix
19 //
20 // Normalized sum of two signals.
21 // The 1/sqrt(2) factor preserves signal energy.
22 //
23 //   nsum(L,R) = (L + R) / sqrt(2)
24 //
25 nsum(x,y) = (x+y)/sqrt(2);
26 //
27 // Normalized difference of two signals.
28 //
29 //   ndif(L,R) = (L - R) / sqrt(2)
30 //
31 ndif(x,y) = (x-y)/sqrt(2);
32 //
33 // Sum-and-difference matrix (Blumlein MS matrix).
34 // 2 inputs (L,R) -> 2 outputs (M,S).
35 //
36 // The matrix is involutory (A = A^-1): applying it twice returns
37 // the original signal. The same operation encodes LR->MS and
38 // decodes MS->LR.
39 //
40 // Energy preservation: |M|^2 + |S|^2 = |L|^2 + |R|^2
41 //
42 // References:
43 // - Alan Blumlein, UK patent 394325 (1933)
44 // - Michael Gerzon, "Surround sound from 2-channel stereo" (1970)
45 //
46 sdmx(x,y) = nsum(x,y), ndif(x,y);

```

```

47 //
48 // Weighted sum-and-difference matrix.
49 // Parameter p controls the M/S balance:
50 // p = 0 -> full Mid (mono)
51 // p = 1 -> full Side (difference only)
52 //
53 sdmxw(p) = sdmx : *(1-p), *(p);
54 //
55 //           Mid-Side panner
56 //
57 // Mono-to-stereo panner using Mid-Side composition.
58 // Synthesizes a virtual stereo microphone pair from a mono signal.
59 // The Mid component has a variable polar pattern (figure-8 to omni),
60 // positive phase facing front. The Side component is a fixed
61 // lateral figure-8, positive phase facing left.
62 // The two components are orthogonal.
63 //
64 // p -- polar pattern control (gain):
65 //     0 = figure-8 (cos(rad), frontal null at 90 degrees)
66 //     0.5 = cardioid
67 //     1 = omnidirectional
68 // rad -- polar pattern angle in radians
69 // x -- mono input signal
70 //
71 // Returns (mid, side) pair. Pass through sdmx to obtain L,R:
72 // mspan_lr(x,p,rad) = mspan(p,rad,x) : sdmx;
73 //
74 // Reference:
75 // Silvi & Tedesco, "SEAM Project -- Sustained Stereophony", SMC 2020
76 //
77 mspan(p,rad,x) = x <: m, s
78 with {
79     m(x) = x*(p) + (x*(1-p)*(cos(rad)));
80     s(x) = x*(sin(rad));
81 };
82 // process = os.osc(1000) : mspan(0.5,ma.PI/2);

```